

PDF Editor — Backend Infrastructure Overview

Audience: Project Manager / non-engineering stakeholders. **Purpose:** Give a complete, plain-language picture of how the backend works, what it can do, what it depends on, and what the operational picture looks like — without diving into individual functions or code.

1. What This Backend Is

The PDF Editor backend is the **engine room** of the product. The browser-based editor users see is the cockpit; this service handles everything that has to happen safely, persistently, or expensively on a server:

- Verifying who the user is and what they're allowed to do
- Storing files securely in the cloud
- Converting PDFs to/from Word, Excel, PowerPoint, images, and other formats
- Running PDF utilities (compress, encrypt, decrypt, watermark, etc.)
- Sending real-time progress updates while large uploads finish
- Recording security-relevant events for audit

The frontend (Next.js web app + admin panel) talks to this backend over HTTPS. The backend never talks back to the user's browser unsolicited — every action is request-driven.

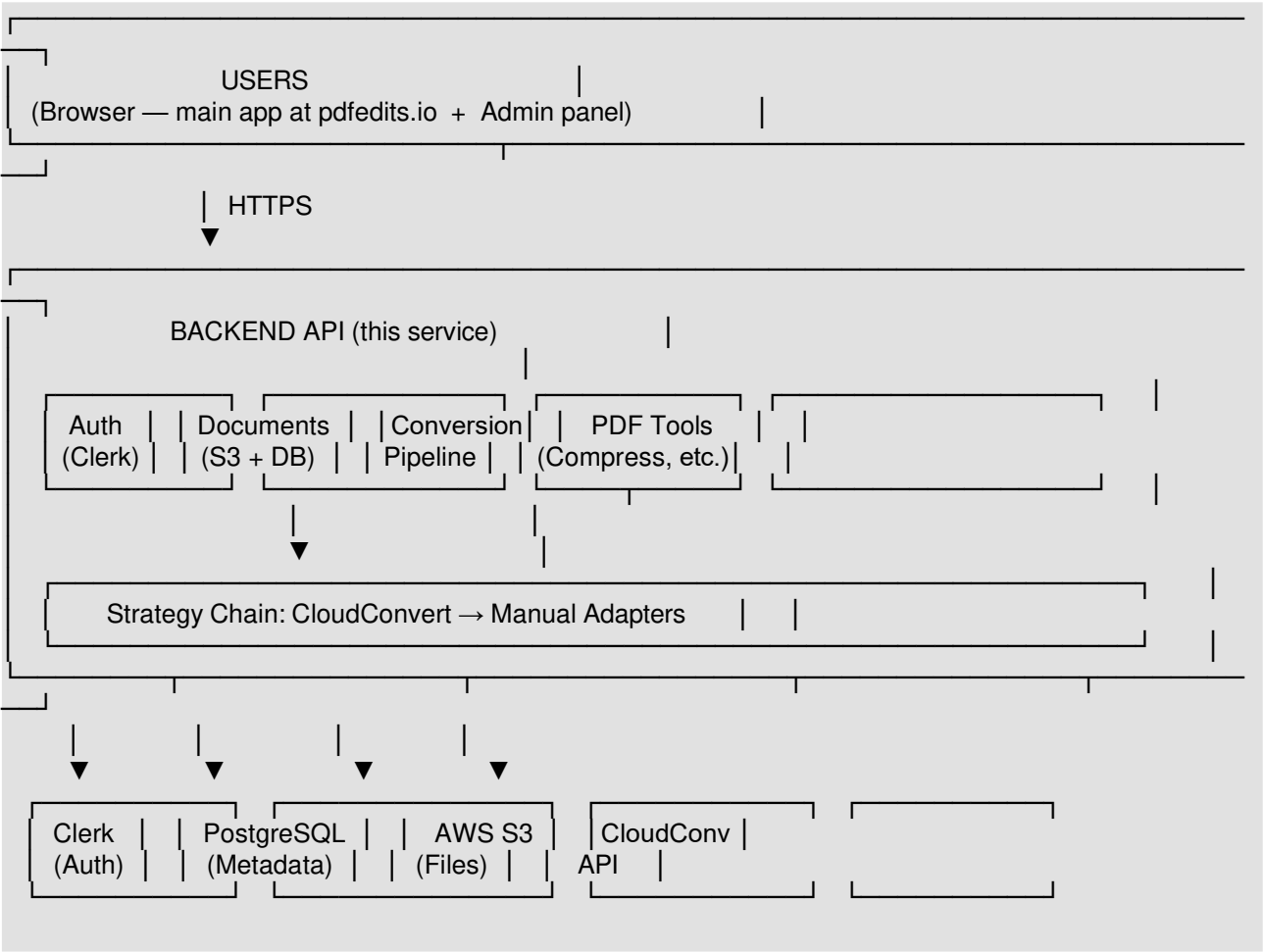
2. Product Capabilities (What End Users Get)

Capability	What It Means For The User
Sign in / Sign up	Email, Google, or Apple login. Handled by Clerk; we just trust the verified identity.
Upload PDFs	Drag-and-drop; large files stream directly to cloud storage with progress feedback.
Document library	Files persist per-user; visible across devices once signed in.
PDF → Office	Convert PDF to Word (DOCX), Excel (XLSX), PowerPoint (PPTX).
PDF → Images	Convert each page to JPG, PNG, WEBP, TIFF, BMP, GIF, SVG, etc. (multi-page → ZIP bundle).
Office → PDF	Convert Word / Excel / PowerPoint back into PDF.

Capability	What It Means For The User
PDF utilities	Compress, password-protect, password-remove, flatten, extract embedded images.
Page operations	Split, rotate, reorder, add watermarks, add page numbers.
Admin panel	Internal staff can view user list, deactivate accounts.

Out of scope (deliberately not built): AI features, real-time collaboration, version history, repair tooling, standalone "merge PDF" page.

3. The High-Level Picture



Plain English: the backend sits in the middle. Users talk to it. It talks to specialised cloud services (Clerk for identity, PostgreSQL for records, AWS S3 for file storage, CloudConvert for high-fidelity conversions) and to local binary tools (LibreOffice, Ghostscript, etc.) running on the same server for fallback / free-tier conversions.

immediately blocked on their next request, even before Clerk knows.

5.2 Documents

- **Upload flow:** the browser asks the backend for a temporary upload link to S3, then sends the file **directly to S3** — bytes never pass through our server. This keeps the API lean and inexpensive.
- **Progress tracking:** while large uploads are happening, the backend can stream live status updates to the browser via Server-Sent Events (SSE).
- **Per-user library:** every file stored is tied to its owner; users only see their own files.
- **Soft delete:** when a user deletes a document, it's marked deleted but retained briefly so admins can recover from accidents.
- **Source of truth:** PostgreSQL stores the metadata (owner, filename, size, type, timestamps); S3 stores the actual bytes.

5.3 Conversion Engine

This is the most complex piece, and the one that has to "just work" across hundreds of format combinations. The design philosophy: **never let one failure kill a request — always have a fallback.**

Strategy chain

When the user requests a conversion, the backend tries strategies in order:

1. **CloudConvert** — paid third-party API. High fidelity. Used first.
2. **Manual adapters** — local open-source tools running on our server. Used when CloudConvert is disabled, doesn't support the format, or fails.

If CloudConvert errors out, the request silently falls through to the manual chain. The user just gets the result.

The manual adapter toolbox

The backend bundles industry-standard PDF / document tools:

Tool	Used For
LibreOffice	Office formats ↔ PDF (DOCX, XLSX, PPTX)
Ghostscript	PDF compression, PDF↔PS conversion
Poppler	PDF → images, PDF text/image extraction
ImageMagick	Image format conversions (PNG/JPG/WEBP/TIFF/etc.)
Calibre	E-book formats, PDF → XLSX intermediate
Pandoc	Universal document converter (markdown, HTML, etc.)

Tool	Used For
Inkscape	Vector formats (SVG, EPS, etc.)
Pstoedit	PostScript → vector formats
qpdf	PDF password protect / unprotect

Each is wrapped in an "adapter" that the backend can substitute, retry, or time out. A central registry knows which tools are installed at boot time and disables strategies whose binaries are missing — so a misconfigured server fails loudly at startup, not silently per request.

Output bundling

For multi-page PDF → image conversions, the backend automatically zips the result into a single download (mydoc-jpg-pages.zip) so the user gets one file containing all pages, not 50 separate downloads.

Safety knobs

- **Hard timeout per strategy** — default 5 minutes; configurable. A stuck conversion can't lock up the server.
- **Concurrency cap** — only N external binaries can run at once across the whole process (default 4). Prevents memory blowouts under load.
- **Per-job temp directory** — each conversion runs in an isolated folder, cleaned up afterwards regardless of success / failure.
- **Kill switch** — CLOUDCONVERT_DISABLED=true forces 100% manual routing if CloudConvert is unreachable or over quota.

5.4 PDF Tools (utility services)

Separate from format conversion, the backend exposes lightweight PDF utilities:

- **Compress** — shrink PDF file size via Ghostscript.
- **Password protect / unprotect** — encrypt or decrypt PDFs via qpdf.
- **Flatten** — bake annotations/form fields into the page image so they can't be edited downstream.
- **Extract images** — pull embedded raster images out of a PDF.

These are simpler (one tool, one job, no fallback chain) but share the same safety knobs — temp isolation, timeout, structured logging.

5.5 Admin / User Management

- GET /users — list all users (admin only).
- Deactivate / reactivate user.
- Audit-log every privileged action.

4. Technology Stack — At A Glance

Layer	Technology	Why
Language / Framework	Node.js + TypeScript + NestJS	Industry-standard, type-safe, modular
Database	PostgreSQL	Reliable, ACID-safe — required for billing & user records
File Storage	AWS S3	Industry standard, infinitely scalable, signed-URL based access
Authentication	Clerk	Managed: email + Google + Apple sign-in, no auth code to maintain
Conversion Engine	CloudConvert + 9 local binaries	Cloud-first with offline fallback chain
Logging	Pino (structured JSON)	Searchable, machine-readable in production
Documentation	Swagger / OpenAPI	Auto-generated API explorer for devs
Hosting	Railway (staging)	Container-based, auto-deploys on git push

The principles are all "boring, proven choices" — nothing exotic. That is deliberate: PDF processing is risky enough; the platform around it should be predictable.

5. Major Subsystems

5.1 Authentication

- **Owned by:** Clerk (third party).
- **Backend role:** verifies each request's JWT token against Clerk's public keys, looks up the corresponding user record in our database, returns 401 Unauthorized when anything is wrong.
- **First sign-in flow:** when a brand-new user signs in, the backend automatically provisions a row in the database for them — no manual setup.
- **Roles:** every user is either a USER (default) or an ADMIN. Admin status can be granted by listing an email in the ADMIN_EMAILS environment setting — the user is promoted automatically on their next sign-in.
- **Account deactivation:** an admin can flip a flag in the database; the user is

5.6 Audit & Security Logging

Every security-relevant event is recorded as a structured log entry:

- Invalid token attempt
- Expired token attempt
- Account deactivation
- Admin promotion
- Webhook receipt

These are searchable in production logs, suitable for forwarding to a SIEM / security dashboard.

6. External Services The Backend Depends On

Service	Purpose	Cost Model	Failure Behaviour
Clerk	Authentication	Per-monthly-active-user	Backend rejects all requests (401).
AWS S3	File storage	Per GB stored + bandwidth	Uploads/downloads fail; rest of API still works.
PostgreSQL	Metadata, users, billing	Per-GB + compute (Railway / Neon)	Backend fully unavailable.
CloudConvert	Premium conversions	Per minute of conversion time	Backend silently falls through to manual chain.
Redis	Caching, job queue (planned)	Per-instance	Read paths unaffected; queue features pause.
Resend <i>(planned)</i>	Transactional email	Per email sent	Outbound mail blocked; rest of API works.
Stripe <i>(planned)</i>	Payments & subscriptions	% of transactions	New subscriptions blocked; existing access unaffected.

Critical dependencies (loss = full outage): **PostgreSQL, Clerk**. Important dependencies (loss = degraded experience): **S3, CloudConvert**.

7. Security Posture

Concern	How It's Handled
Authentication	Every request requires a valid Clerk-signed JWT. Tokens are verified against Clerk's public keys with strict algorithm checks (prevents the classic "alg=none" attack).
Authorization	Role-based: admin-only endpoints reject normal users with HTTP 403.
File access	S3 buckets are private. Files are accessed only via short-lived signed URLs (≤ 15 minutes).
Data in transit	HTTPS everywhere. No plain HTTP in production.
Data at rest	S3 server-side encryption. Database uses TLS + storage-level encryption.
Cross-origin requests (CORS)	Locked to known frontend / admin origins. Localhost auto-allowed in development only.
Secrets	All API keys / credentials live in environment variables. None checked into source code. The application refuses to start if a required secret is missing.
Audit trail	Every authentication failure and privileged action is logged with timestamp, IP, and severity.
No secret leakage	Logs and error messages are scrubbed of tokens, passwords, and personal data.
Input validation	Every API request body is validated against a typed schema; rejected requests never reach business logic.

8. Reliability Practices

- **Stateless server.** Any instance can serve any request. Scaling horizontally is just adding more containers — no sticky sessions.
- **Graceful conversion fallback.** Strategy chain ensures that a CloudConvert outage or a single missing binary does not break the feature.
- **Hard timeouts** on every external call and conversion strategy.
- **Resource cleanup** is guaranteed (per-job temp folders deleted even on failure).

- **Boot-time validation** of environment variables. Configuration mistakes surface immediately at deploy, not at 3 AM under load.
- **Structured logs** mean an on-call engineer can grep for requestId=... and see the full path of any failed request in seconds.

9. Performance & Scale

- **API process stays lean.** All heavy work (PDF conversion, compression) runs in subprocesses or external services. The main Node.js process never blocks on expensive work.
- **Files are streamed**, never buffered. A 200 MB upload doesn't consume 200 MB of server RAM.
- **Direct-to-S3 uploads** mean the backend never proxies file bytes.
- **Concurrency control** caps total simultaneous CPU-heavy conversions per instance.
- **Database access is paginated** and indexed for the queries we actually run.

Today's scale target: comfortably serves hundreds of concurrent users on a single small Railway instance. Scaling beyond that is a configuration change, not an architectural one.

10. Environments

Environment	URL	Purpose
Local dev	http://localhost:7403	Engineer's machine for development.
Staging	https://pdf-viewer-backend-staging.up.railway.app	Pre-production for QA / PM review.
Production	(TBD when launched)	End-user-facing.

Frontends:

Environment	Main App	Admin Panel
Staging	https://pdf-viewer-app-staging.up.railway.app	https://pdf-viewer-admin-staging.up.railway.app
Production	https://www.pdfedits.io	(TBD)

Each environment has its own database, S3 bucket, Clerk instance, and CloudConvert account — so staging activity never touches production data.

11. Configuration (Environment Variables)

Configuration is **never** hard-coded — every secret, URL, and tuning knob is an environment variable. Adding a new environment is a matter of supplying the right values, not changing code.

Categories:

- **App basics** — port, environment name, allowed CORS origins.
- **Database** — Postgres connection string.
- **Authentication** — Clerk publishable key, secret key, JWKS URL, webhook secret, optional admin email allow-list.
- **AWS S3** — region, access key, secret key, bucket name.
- **Conversion engine** — CloudConvert API key + sandbox flag, timeout, max concurrent binary executions.
- **Optional** — Google / Microsoft OAuth IDs (for cloud import features).

Every variable is documented in `.env.example`. The application refuses to start if any required one is missing — eliminating "it works on my machine" class of bugs.

12. Deployment & Release

- **Hosting:** Railway (containerised Node.js).
 - **Trigger:** push to the relevant branch → CI builds → Railway redeploys.
 - **Database migrations:** managed via Prisma. Migrations are version-controlled and applied automatically on deploy.
 - **Build pipeline:** type-check, lint, unit tests, then compile.
 - **Rollback:** Railway keeps prior deploys; reverting is one click.
-

13. Observability — How We Know What's Happening

- **Structured logs** in JSON, every request tagged with method, URL, status, duration, user ID, and a correlation ID that follows the request through every internal layer.
- **Boot logs** announce environment-level configuration (issuer, admin count, allowed CORS origins) so any misconfiguration is visible immediately.
- **Audit events** carry severity (low / medium / high) so dashboards can surface what matters.
- **Swagger / OpenAPI** docs exposed at `/api/docs` in non-production environments — engineers can click around the API live.

Planned for production hardening:

- Sentry for error tracking
 - OpenTelemetry / Datadog metrics
 - Long-term log retention for compliance
-

14. Current Status & Roadmap

The backend already covers everything needed for the core editor product:

✓ Done

- Clerk-based authentication, role-based access control
- Document upload (direct-to-S3), listing, download, delete
- Server-sent live upload progress
- PDF ↔ DOCX / XLSX / PPTX conversion
- PDF → image formats (JPG, PNG, WEBP, TIFF, BMP, GIF, SVG, etc.) with ZIP bundling for multi-page output
- PDF utilities: compress, encrypt, decrypt, flatten, extract images
- Admin user list / deactivation
- CloudConvert primary + manual fallback chain (9 adapters)
- Audit logging for security events
- Structured logging with correlation IDs
- Environment validation on boot

🚧 In progress / planned

- Stripe billing (trial \$0.70 / 7 days → \$19.99/mo) — schema and structure in place; webhook + checkout flow pending.
- BullMQ background job queue — for long-running conversions to be moved off the request thread.
- Transactional email (Resend) — welcome, password reset, conversion-complete.
- Sentry + metrics integration.
- Per-user / per-plan usage quotas.
- S3 lifecycle policies for cost control on old / deleted files.

The architecture was designed with these in mind from day one, so adding them is incremental — no rewrites required.

15. Risks & What We're Doing About Them

Risk	Mitigation
CloudConvert outage / price spike	Local adapter fallback chain is feature-complete; the product keeps working.
Cost runaway from large file uploads	Per-file size limit enforced server-side; concurrency cap on conversions.
Stuck conversion locking server	Hard timeouts on every strategy.
Lost files	S3 is replicated by AWS by default; soft-delete grace period before hard delete; lifecycle policies prevent accidental purge.
Auth provider compromise	Tokens are short-lived; account deactivation flag in our DB lets us cut off any user immediately, independent of Clerk.
Misconfigured deploy	Environment validation fails the boot; bad deploys never start serving traffic.
CORS misconfiguration locking out frontend	Origin list is env-driven for prod, but a developer-only safety net auto-allows localhost.

16. Glossary

Term	Meaning
API	The set of HTTP endpoints the frontend calls.
JWT	A signed token Clerk gives the user that proves who they are.
CORS	Browser security: an API can declare which websites are allowed to call it.
SSE	Server-Sent Events — a way for the backend to push live updates to the browser over a normal HTTP connection (used for upload progress).
S3	Amazon's file storage service. Files live there; metadata lives in PostgreSQL.
Adapter / Strategy	Software design pattern: every conversion tool implements the same interface so the engine can swap them in/out without

Term	Meaning
	caring which one runs.
Presigned URL	A temporary, single-purpose link to upload or download a specific S3 file; expires in minutes.
JWKS	The published list of Clerk's public keys we use to verify their JWTs.
Audit log	A separate, security-focused log of who-did-what for compliance / forensics.
Soft delete	Marking a record deleted in the database without immediately removing it, so recovery is possible.

This document describes the backend as it stands today. For function-level details, see the source code and documents-api.md.